Bahria University, Islamabad Campus

Department of Software Engineering
Course Instructor: Engr. Saad Mazhar Khan
Course: Artificial Intelligence
Class/Section: BSE-6 (A/B)
Paper Type: OEL [CLO-2]

Integration and Deployment of a Multi-Model Agricultural Intelligence System

Case Study & Problem Statement

Modern precision agriculture demands integrated decision-support platforms that transcend isolated algorithmic experiments. A national agri-tech consortium requires a unified Smart Agriculture Decision Support System capable of synthesizing heterogeneous data sources into coherent, actionable intelligence for farm managers and agronomists.

The consortium has specified that the solution must leverage classical machine learning methodologies specifically Decision Tree Classification, K-Nearest Neighbors Clustering, and Linear Regression as core algorithmic modules. These components have been pre-validated for interpretability and computational efficiency in resource-constrained deployment environments.

You are required to **ASSEMBLE** these disparate algorithmic modules into a single, cohesive, production-grade software system. The final deliverable must demonstrate your ability to integrate multiple AI components into a unified pipeline, interface with end-users through a graphical application, and present the solution with industry-standard documentation suitable for both technical stakeholders and research peer review.

Lab Objectives

Upon successful completion of this OEL, the student shall be able to:

1. **ASSEMBLE** a multi-model AI pipeline by integrating classification, clustering, and regression paradigms into a single executable architecture.
2. **Construct** a modular software solution with clean separation of concerns across the data layer, model layer, and presentation layer.
3. **Calibrate** and validate each constituent model using appropriate performance metrics, ensuring the assembled system produces reliable, reproducible outputs.

4. **Organize** and document the complete system using modern development tools including version control, dependency management, and technical writing frameworks to produce publication-ready artifacts.

Technical Requirements & System Specifications

1. Data Engineering Layer

Acquire a real-world agricultural dataset (public repositories such as Kaggle, UCI ML Repository, or government open-data portals are permissible). Execute comprehensive preprocessing, including but not limited to: missing value imputation, outlier treatment, feature scaling, and categorical encoding. Document the data dictionary and preprocessing rationale.

2. Algorithmic Core

Implement and train the following three mandatory components:

- **Decision Tree Classifier:** Crop recommendation based on soil and environmental features. Report accuracy, precision, recall, and display the feature importance vector.
- **KNN Clustering Model:** Soil profile segmentation to identify homogeneous farm zones. Report silhouette scores and visualize cluster distributions.
- **Linear Regression Model:** Crop yield prediction (quantitative output). Report RMSE, MAE, and R^2, accompanied by residual analysis plots.
- Serialize all trained models using **joblib** or pickle for inference reuse.

3. System Assembly & Interface Layer

Using Tkinter (leveraging prior lab competencies), ASSEMBLE a unified Graphical User Interface that binds the three serialized models into a single interactive application. The interface must:

- Accept user inputs for relevant soil and climatic parameters.
- Invoke the serialized models sequentially and display integrated outputs: recommended crop type, assigned soil cluster with agronomic guidance, and predicted yield with confidence bounds.
- Embed matplotlib visualizations directly within the GUI frame (minimum three distinct plots: feature importance, cluster scatter, and residual plot).

4. Repository & Documentation

Establish a public GitHub repository adhering to professional software engineering standards:

```
repository/
├── data/              # Dataset and metadata
├── src/               # Modular source code
│   ├── preprocessing.py
│   ├── models.py
│   ├── gui.py
│   └── utils.py
├── models/            # Serialized model artifacts
├── results/           # Evaluation metrics and screenshots
├── requirements.txt   # Exact dependency manifest
├── LICENSE            # Open-source license (MIT/Apache 2.0)
└── README.md          # Comprehensive project documentation
```

The README.md must contain: system architecture diagram, installation and execution instructions, algorithmic rationale, quantitative performance summary, and a "Future Work" section proposing two concrete research extensions.

5. Technical Report

Submit a formal typeset report (4–6 pages, IEEE or ACM format preferred) comprising:

- **Abstract:** Concise summary of the problem, methodology, and key results.
- **Introduction:** Problem domain significance and related work (minimum three citations).
- **Methodology:** Data pipeline, model selection rationale, integration strategy, and GUI design decisions.
- **Results & Discussion:** Comparative performance metrics, visualization interpretation, and system limitations.
- **Industrial Application:** A clear exposition of how this assembled system translates to commercial agri-tech deployment.
- **Research Extension:** Two academically viable directions for future investigation (e.g., IoT sensor integration, ensemble deep learning, or satellite imagery fusion).
- **Conclusion:** Synthesis of engineering outcomes and professional reflections.

Constraints & Regulations

- Full internet access is permitted for library acquisition and tooling; however, the three specified algorithms (Decision Tree, KNN, Linear Regression) must remain the primary analytical engines.
- All code must be original. Plagiarism detection will be applied to both source code and technical reports.
- The GitHub repository must be public and remain accessible for a minimum of one academic semester.

Assessment Rubric

Criterion	Weight	Descriptor
System Assembly & Integration	25%	Correct binding of multiple models into a unified pipeline; GUI stability; seamless component interaction; proper model serialization and inference.
Algorithmic Rigor & Evaluation	20%	Proper training/testing protocols; relevant metrics for each model; meaningful visualizations; calibrated outputs.
Repository Quality & Documentation	20%	Modular architecture; professional README; clean commit history; dependency management; open-source compliance.
Technical Report Quality	20%	Academic writing standards; clear industrial application narrative; viable research extensions; proper citations.

Submission Protocol

Submit the following via the designated Learning Management System (LMS) portal before the stated deadline:

1. GitHub Repository URL (public access verified).
2. Technical Report (PDF format).
3. results/ folder compressed archive containing all evaluation plots and screenshots.